实验一：基于 RISC-V 指令系统与 HBirdv2/DDR200T 平台初体验

课程：计算机组成原理
参考教材：William Stallings, Computer Organization and Architecture: Designing for Performance, 10th Ed.

一、实验目的

学生完成本实验后应能：
1．了解 RISC-V 通用寄存器的命名与基础功能（x0-x31, a0, ra, sp 等）。
2．在 DDR200T 开发板上，使用 Nuclei Studio 新建、编译、下载并运行一个“Hello World”程序。
3．熟悉 Nuclei Studio 的 LST 视图，对照 C 代码阅读 RISC-V 指令。
4．熟悉并编写一个简单的纯汇编子程序（例如 sum_to_n），理解 RISC-V 调用约定。

二、预习要求

1．教材复习

（1）参考教材第 13-15 章：指令系统、寻址方式、指令类型。
（2）参考教材第 16 章：指令周期、取值-执行模型。

2．资料预习

（1）阅读 HBirdv2 SoC 文档中“Core Overview”和“GPIO/UART 简介”部分，了解 E203 是一颗两级流水的 RV32IMAC 核。
（2）阅读 HBird SDK Quick Start 的“Hello World with Nuclei Studio”小节，知道创建/编译/下载的基本流程。

3．预习思考题

（1）RISC-V 共有多少个整数寄存器？x0 有什么特殊性？
（2）调用一个子程序时，返回地址保存在哪个寄存器？返回值通常放在哪个寄存器？
（3）比较参考教材中描述的典型 RISC 特征，与 RISC-V ISA 的对应关系。

三、实验环境与设备

1. 硬件

（1）Nuclei DDR200T 开发板（板上含 HBirdv2 E203 SoC）。
（2）JTAG 调试器。

2. 软件

（1）Nuclei Studio，需安装 Nuclei RISC-V 工具链与 HBird SDK NPK 插件。
（2）下载地址在：https://www.nucleisys.com/download.php 选中 Nuclei Studio IDE 2025.10 (win10/11) 下载。

四、实验内容与任务概览

1. 任务 1：搭建环境并运行 Hello World

（1）在 Nuclei Studio 中创建/导入工程中。
（2）成功下载到板上，在串口看到“Hello World From RISC-V Processor!”等输出。

2. 任务 2：观察 C 代码对应的 RISC-V 指令

（1）在 main.c 中加入若干算术/逻辑语句。
（2）使用 LST/反汇编视图观察对应指令，记录若干条指令与寄存器变化。

3. 任务 3：编写并调用纯汇编子程序 sum_to_n

（1）新建 sum.S 文件，实现 uint32_t sum_to_n(uint32_t n); 的 RISC-V 汇编循环。
（2）在 main.c 中 extern 声明并调用该函数，打印结果。
（3）用调试器单步进入汇编函数，观察 a0、t0、t1、ra 的变化，理解 RISC-V 调用约定。

五、实验步骤

1. 任务 1：创建并运行 Hello World

（1）启动 Nuclei Studio，选择工作空间（workspace）。

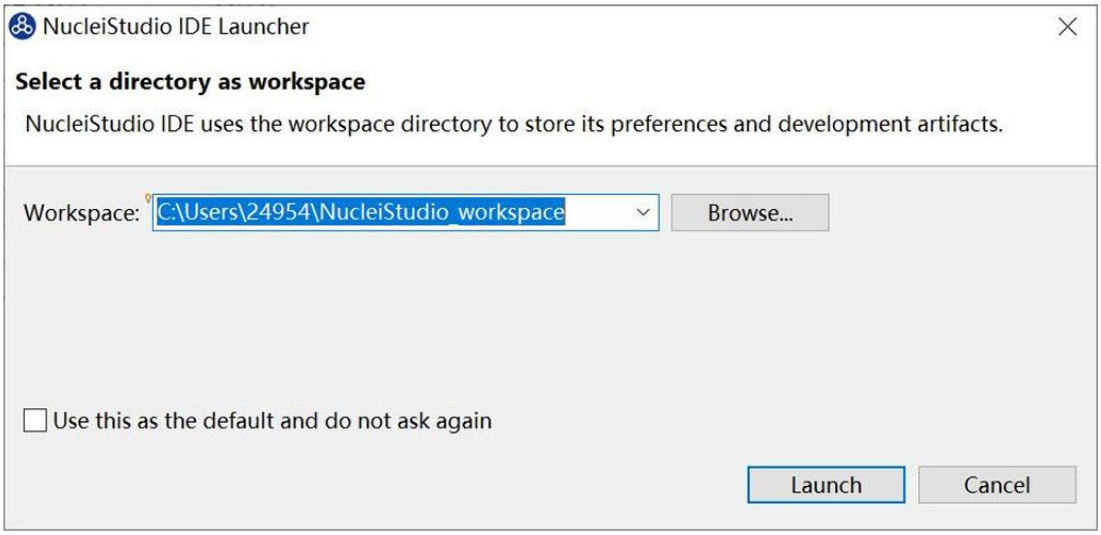

图 1 启动 Nuclei Studio，选择 workplace

（2）下载 HBirdv2 工程所需 NPK 软件包。

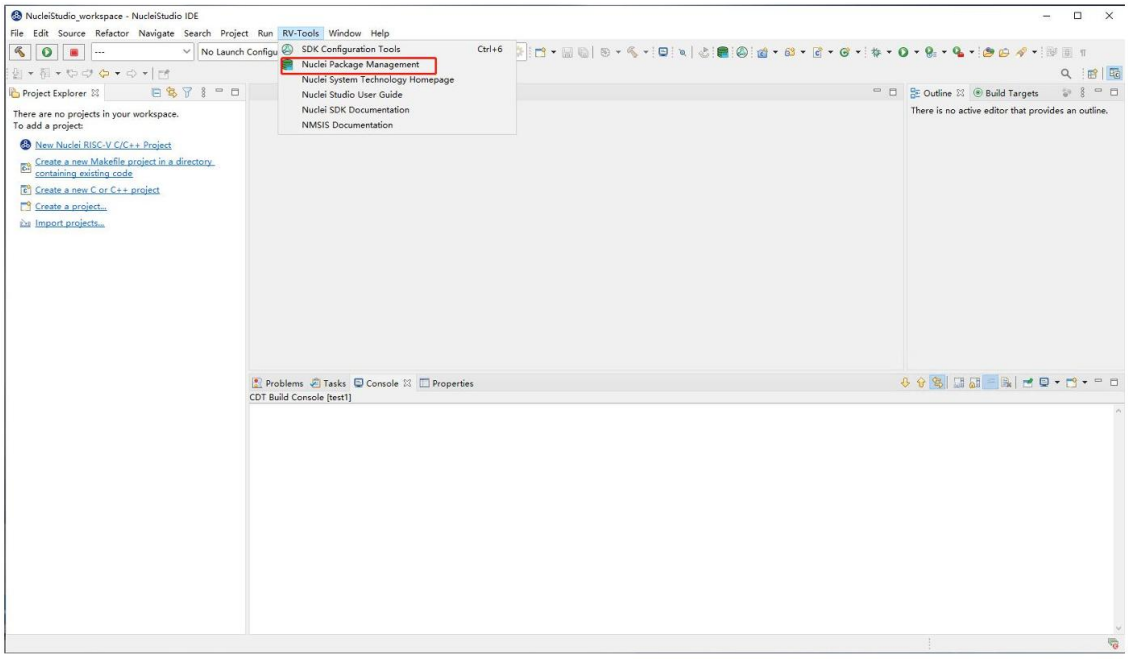

图 2 选中 RV-Tools 中的 Nuclei Package Management

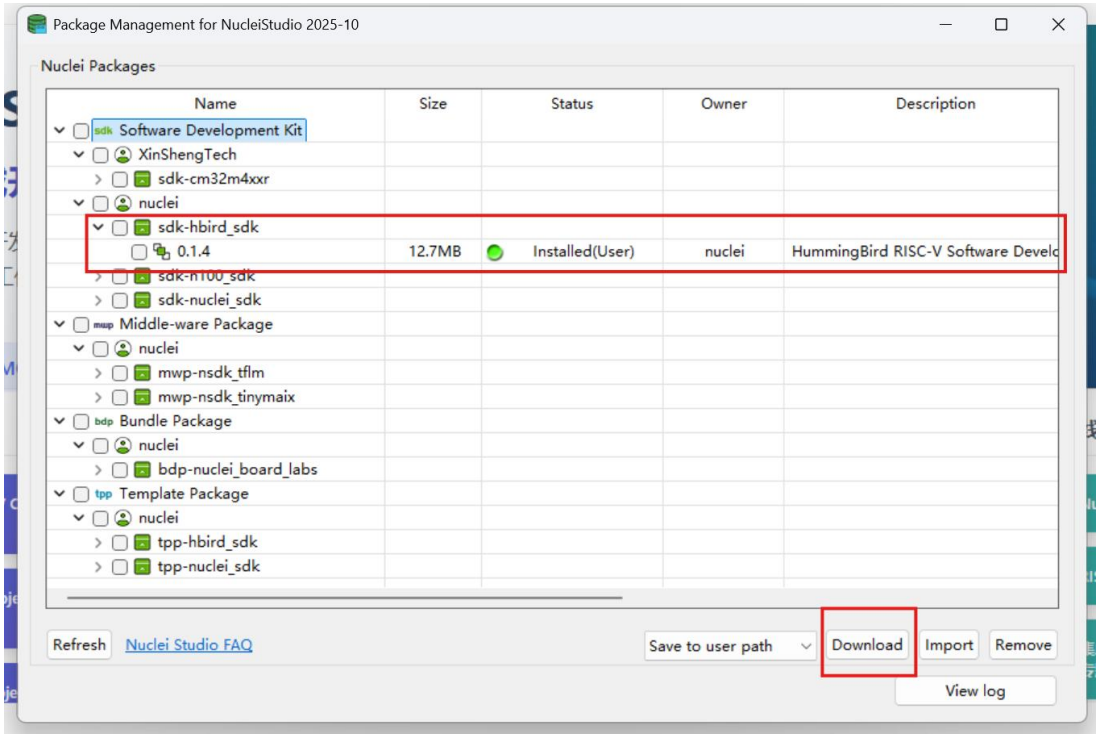

图 3 选中 nuclei -> sdk-hbird_sdk -> 0.1.4，并 Download

（3）新建 HBirdv2 工程，使用示例 Hello World，目标开发板选择 DDR200T。

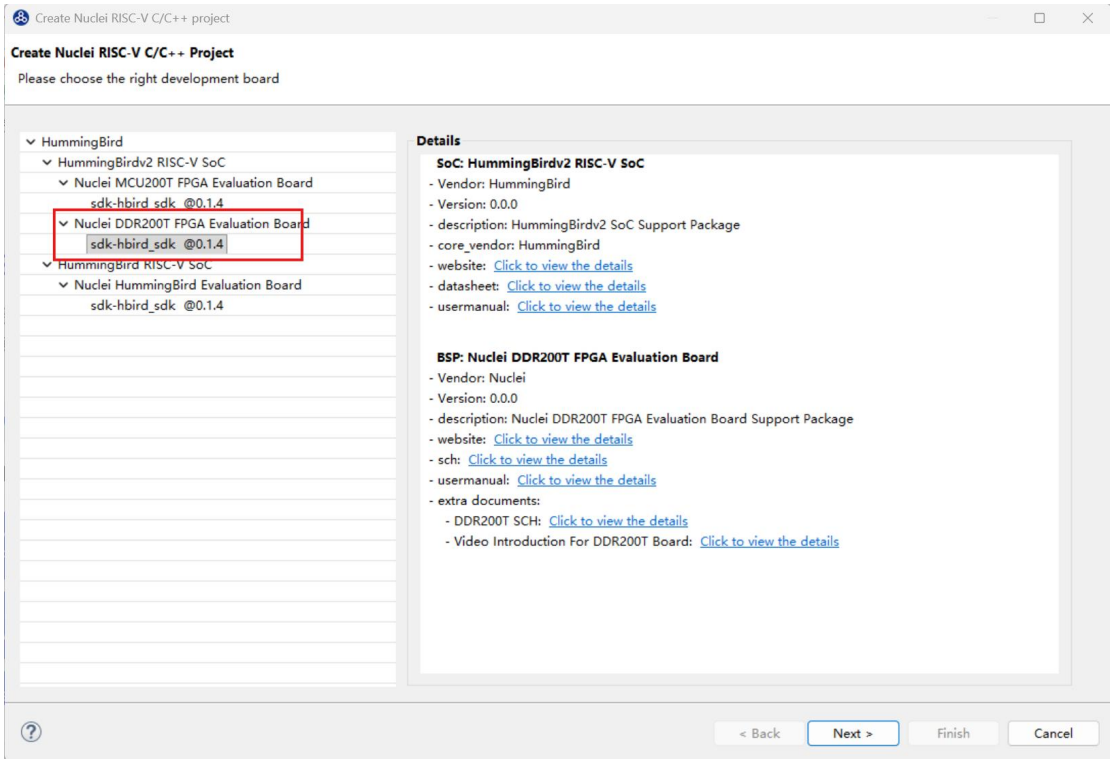

图 4 选中 Nuclei DDR200T FPGA Evaluation Board

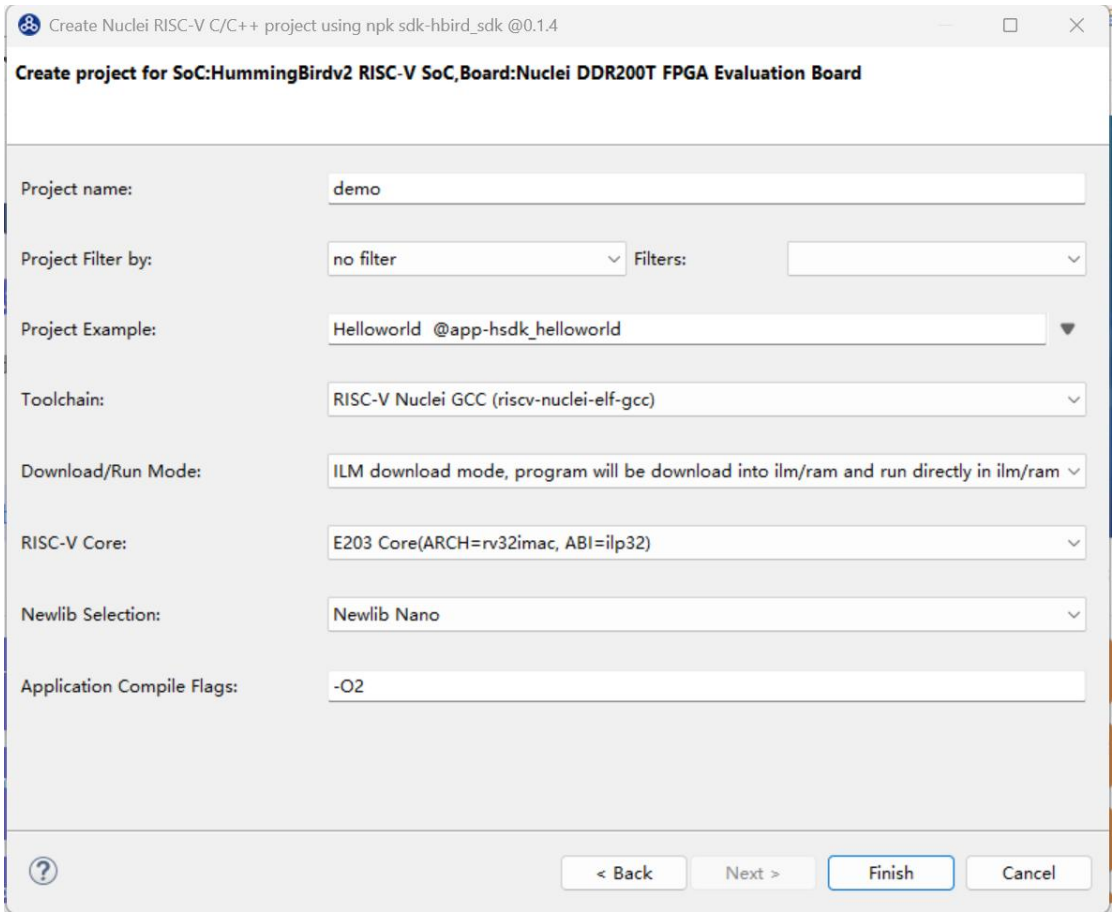

图 5 创建 Helloworld 示例工程文件

（4）将工程编译链从 gcc 10 转换成 gcc 13。

在工具栏中选中 Project -> Convert All -> Convert All gcc10 Project to gcc13

（5）编译工程（Build），确认无错误。

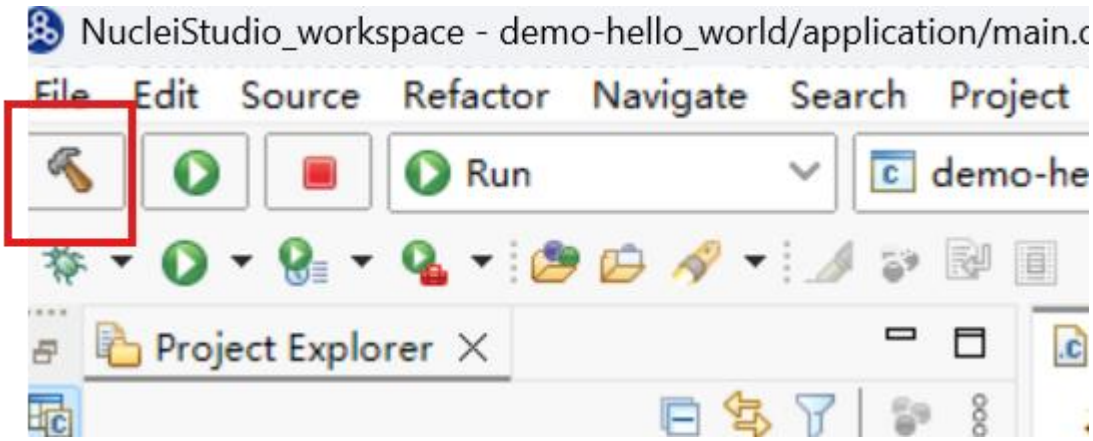

图 6 选中 Debug 标志

（6）连接开发板，配置串口号和波特率（通常为 115200bps）。

在工具栏中选中 Window -> Show View -> Terminal

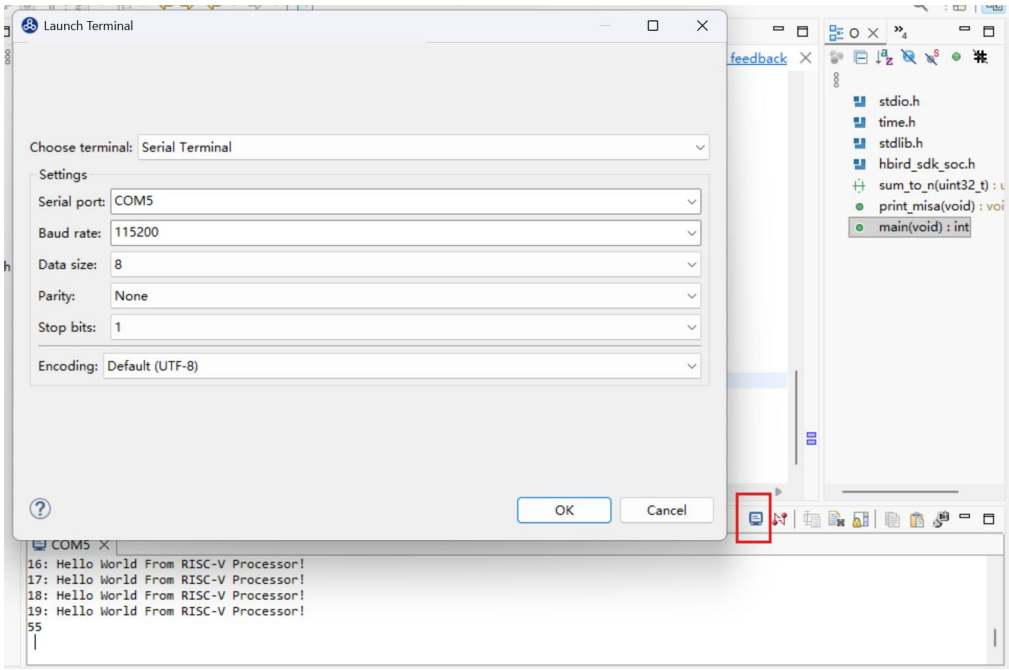

图 7 选中 Terminal 配置选项并进行配置

注意：Serial ports 编号需要在设备管理器中查询，各个机器可能不同。

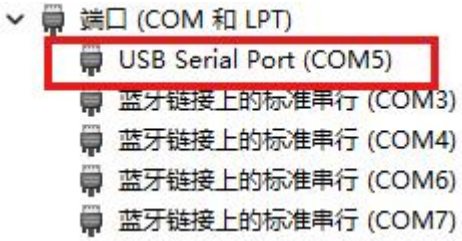

图 8 USB Serial Port COM 编号

（7）在 Nuclei Studio 中启动 Run，下载程序到板上。
（8）在串口终端中截取程序输出，留存实验记录。

2. 任务 2：通过 LST 视图观察汇编

（1）在 main.c 中添加简单代码，如下：

```
int a = 3, b = 5;
int c = a * b + 7;
```

（2）重新编译工程。
（3）打开项目生成的.lst 文件，文件存放在 Debug 文件夹中。
（4）找到上述 C 语句对应的 RISC-V 指令段，记录至少 5 条指令，以及它们使用的寄存器。
（5）任选一条指令，用调试器单步执行，填写“执行前/执行后”寄存器值表格。

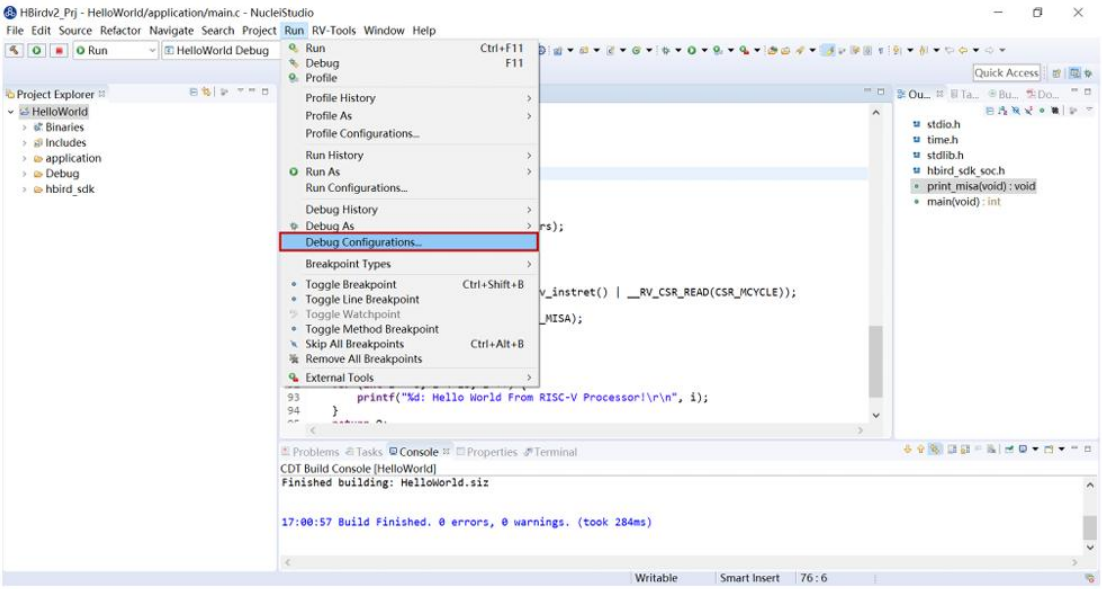

图 9 在工具栏，选择“Run->Debug Configuration”

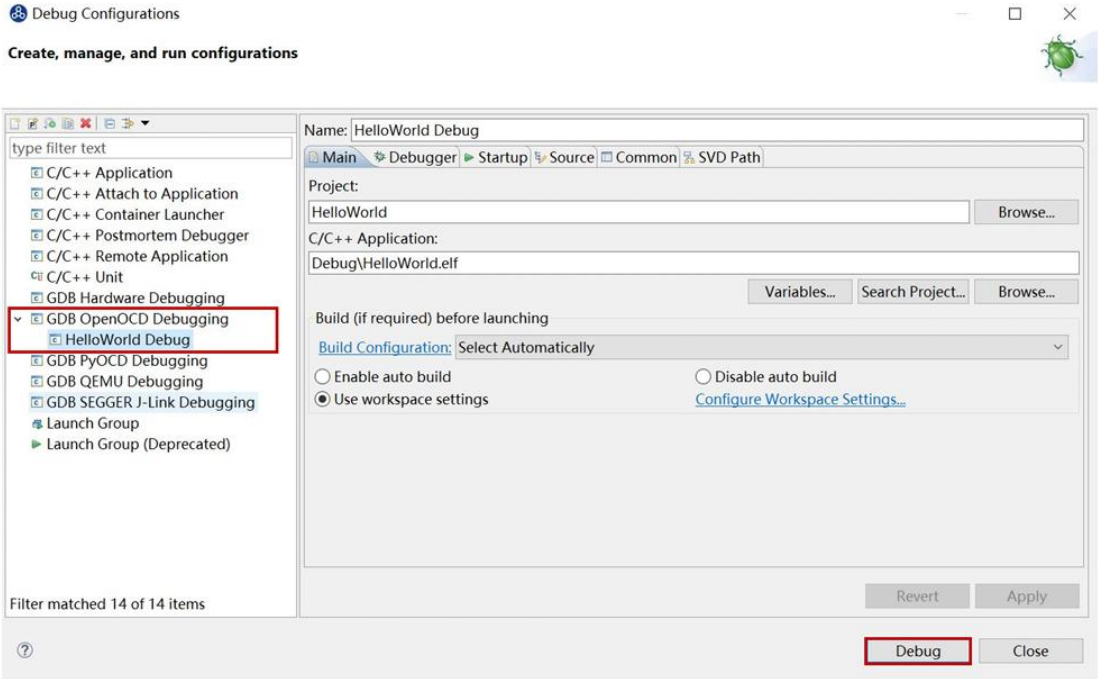

图 10 GDB OpenOCD Debugging 配置

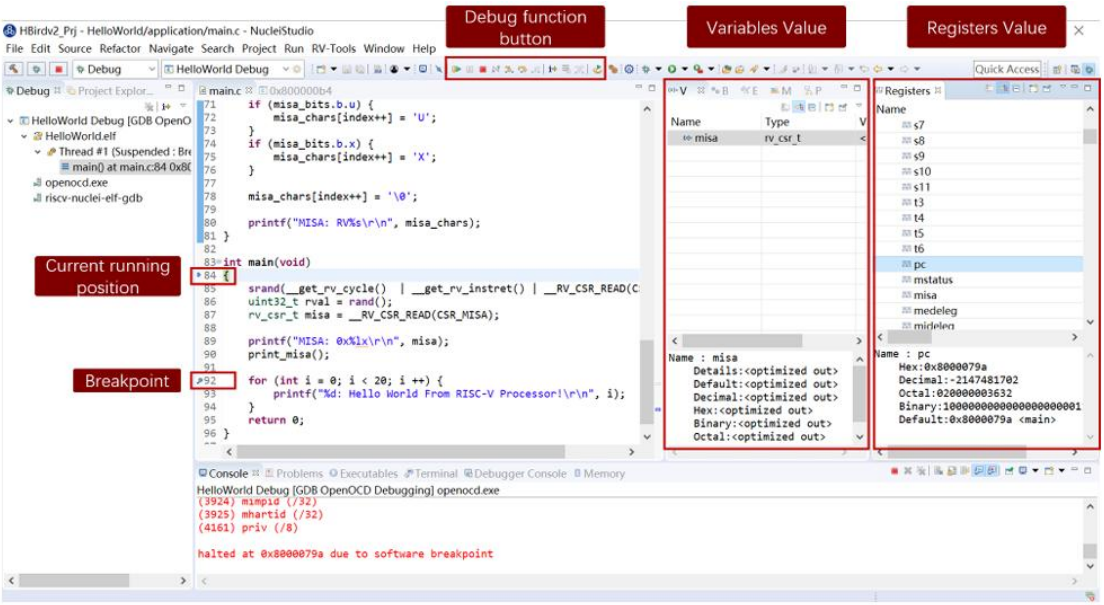

图 11 Debug 使用方法

3. 任务 3：编写纯汇编子程序

（1）在工程中新建源文件 sum.S，文件内容示例骨架：

```
.section .text
.global sum_to_n

# uint32_t sum_to_n(uint32_t n);
sum_to_n:
    # TODO: complete your code here.
    ret
```

（2）查阅 RISC-V 调用，如附录所示

（3）补全为循环求和版本（1+2+...+n）。

（4）在 main.c 开头添加声明：

```
extern uint32_t sum_to_n(uint32_t n);
```

（5）在 main() 中调用

```
uint32_t s = sum_to_n(10);
printf("sum_to_n(10) = %u\r\n", s);
```

（6）编译下载，确认输出 sum_to_n(10) = 55。

（7）使用调试器：在调用处打断点，单步进入 sum_to_n；观察 a0、t0、t1 的变化，画出循环执行过程。

六、实验记录与报告要求

1．必须提交的截图/记录

（1）Nuclei Studio 工程结构和成功编译界面；
（2）串口终端中 Hello World 和 sum_to_n 输出；
（3）至少一张 LST View 中 C 语句（sum_to_n）对应的 RISC-V 指令截屏；
（4）调试器中查看寄存器的截图（执行 sum_to_n 前后对比）。

2．需要填写的表格建议

（1）指令-寄存器变化表

指令地址	指令文本	执行前寄存器	执行后寄存器	说明
0xXXXX	Add t0, t0, t1	t0=?, t1=?	t0=?,t1=?	t0 累加，t1 不变

3．文字分析

（1）概述 E203 所支持的 RISC-V ISA 组合（例如 RV32IMAC），并解释各扩展含义。
（2）结合参考教材中提到的 RISC 特征，总结观察到的 RISC-V 指令特点（固定长度、load/store、简单寻址方式等）。
（3）用你调试 sum_to_n 的记录，说明 RISC-V 调用约定如何工作；参数/返回值在哪些寄存器，ret 实际做了什么。

七、思考题

1．请根据你的 LST 文件的观察，比较 C 语句 c = a * b + 7; 在 RISC-V 上生成的指令序列，与参考教材中其他 ISA（如 x86 或 ARM）示例在思想上的相似之处。
2．若将 sum_to_n 改写成递归版本（sum_to_n(n) = n + sum_to_n(n-1)），RISC-V 的栈（sp）和返回地址（ra）会如何变化？
3．在 E203 简单的两级流水线（取值/执行）模型中，sum_to_n 的循环部分是否存在数据相关与分支相关？请结合参考教材中第 16 章给出的流水线图，做定性分析即可。

八、附录

1．RISC-V 寄存器速查表

硬件名	ABI 名	编号	典型用途
x0	zero	0	常数 0，读总是 0，写入被丢弃
x1	ra	1	return address，子程序返回地址

x2	sp	2	stack pointer，栈顶指针
x3	gp	3	global pointer，全局数据指针
x4	tp	4	thread pointer，线程本地数据指针
x5	t0	5	temporary，调用者保存 (caller-saved)
x6	t1	6	temporary，caller-saved
x7	t2	7	temporary，caller-saved
x8	s0/fp	8	saved register 0 / frame pointer，被调用者保存 (callee-saved)
x9	s1	9	saved register 1，callee-saved
x10	a0	10	argument 0 / return value 0，函数第 1 个参数/返回值
x11	a1	11	argument 1 / return value 1，函数第 2 个参数/第二返回值
x12	a2	12	argument 2，函数第 3 个参数
x13	a3	13	argument 3
x14	a4	14	argument 4
x15	a5	15	argument 5
x16	a6	16	argument 6
x17	a7	17	argument 7 / 系统调用号（ecall）
x18	s2	18	saved register 2，callee-saved
x19	s3	19	saved register 3，callee-saved
x20	s4	20	saved register 4，callee-saved
x21	s5	21	saved register 5，callee-saved
x22	s6	22	saved register 6，callee-saved
x23	s7	23	saved register 7，callee-saved
x24	s8	24	saved register 8，callee-saved
x25	s9	25	saved register 9，callee-saved
x26	s10	26	saved register 10，callee-saved
x27	s11	27	saved register 11，callee-saved
x28	t3	28	temporary，caller-saved
x29	t4	29	temporary，caller-saved
x30	t5	30	temporary，caller-saved
x31	t6	31	temporary，caller-saved

调用约定要点：

- 参数用 a0–a7 传递，返回值在 a0（必要时 a1 做第二返回值）；
- t0–t6 由调用者保存（caller-saved），子程序可以随便用；
- s0–s11 由被调用者保存（callee-saved），如在子程序中修改，需要先入栈、返回前恢复；
- ra（x1）是 jal/jalr 保存返回地址用的，ret 本质上是 jalr x0, ra, 0。

2. 常用 CSR 的简单说明

类别	CSR 名称	地址（hex）	读写	作用

机器模式状态	mstatus	0x300	R/W	全局中断使能位 MIE、异常返回前的状态 MPIE/MPP 等
ISA 描述	misa	0x301	RO	描述本核支持的 XLEN 和扩展，如 RV32 IMAC
中断使能	mie	0x304	R/W	各类 Machine 中断的“是否允许”
周期计数	mcycle	0xB00	R/W	64 位 Machine 周期计数器的低 32 位
指令条数	minstret	0xB02	R/W	64 位“Retired 指令数”的低 32 位
厂商 ID	mvendorid	0xF11	RO	厂商编号（比如 Nuclei / OpenHW 的 JEDEC ID）
架构 ID	marchid	0xF12	RO	表示处理器架构系列（比如某个内核家族 ID）
实现 ID	mimpid	0xF13	RO	具体实现/版本号，用来区分同一架构的不同版本

3. 参考资料列表：

（1）Stallings, Computer Organization and Architecture 10/e.

（2）HBirdv2 SoC Documentation (https://doc.nucleisys.com/hbirdv2/hbirdv2_soc.pdf)

（3）HBird SDK Quick Start (https://doc.nucleisys.com/nuclei_sdk/develop/index.html)

（4）Nuclei Studio User Guide (https://www.nucleisys.com/upload/files/doc/nucleistudio/Nuclei_Studio_User_Guide.202402.pdf)